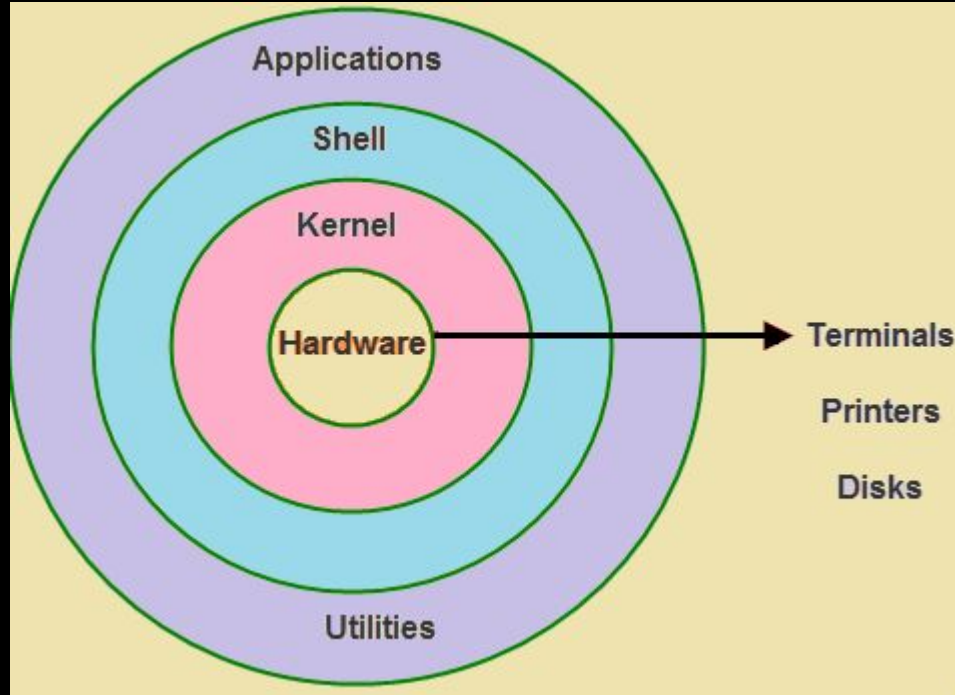


System Calls

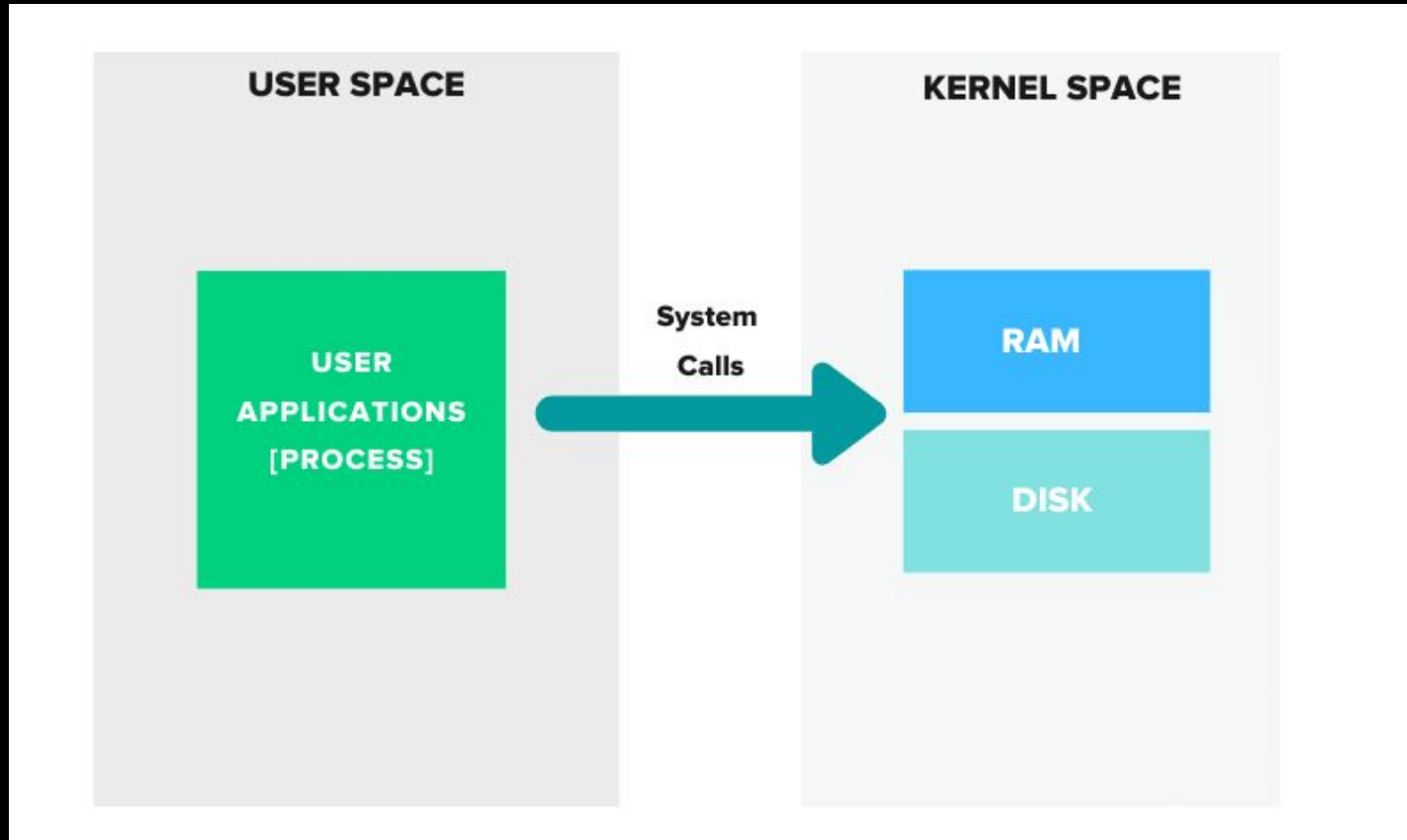
System Calls

- System calls are the **programmatic interface** that allows a user application to request services directly from the operating system kernel.
 - They act as a secure bridge, letting software **interact safely with physical hardware** and core system resources without risking system stability.
 - To protect the system, modern **operating systems divide execution into two spaces**:
 - **User Mode**: *Restricted environment where standard applications run.*
 - **Kernel Mode**: *Privileged environment with unrestricted access to system hardware.*
 - When a program needs a resource (like reading a file), it executes a specific trap or software interrupt. This **switches the CPU from User Mode to Kernel Mode**, executes the requested service safely inside the kernel, and then switches back
-
- **Interrupt** is a signal sent to a computer's processor by hardware or software demanding immediate attention

Shell & Kernel

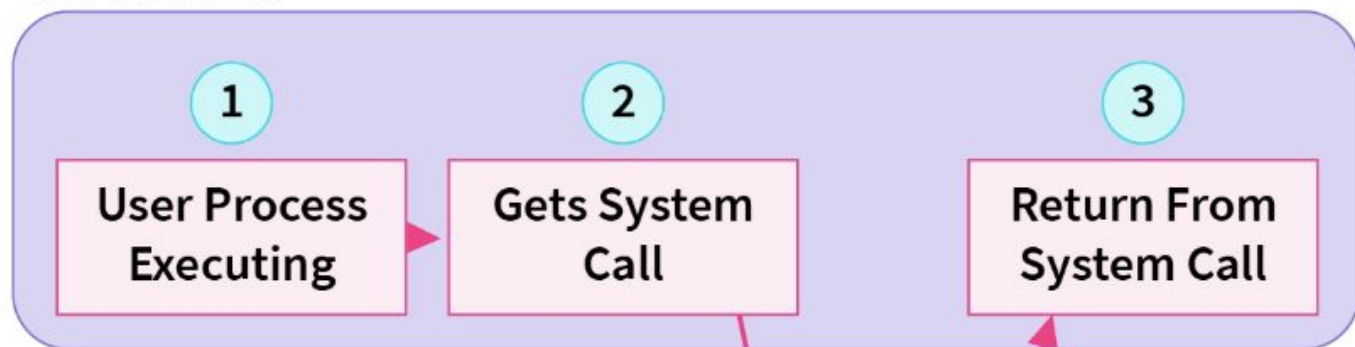


User Mode & Kernel Mode

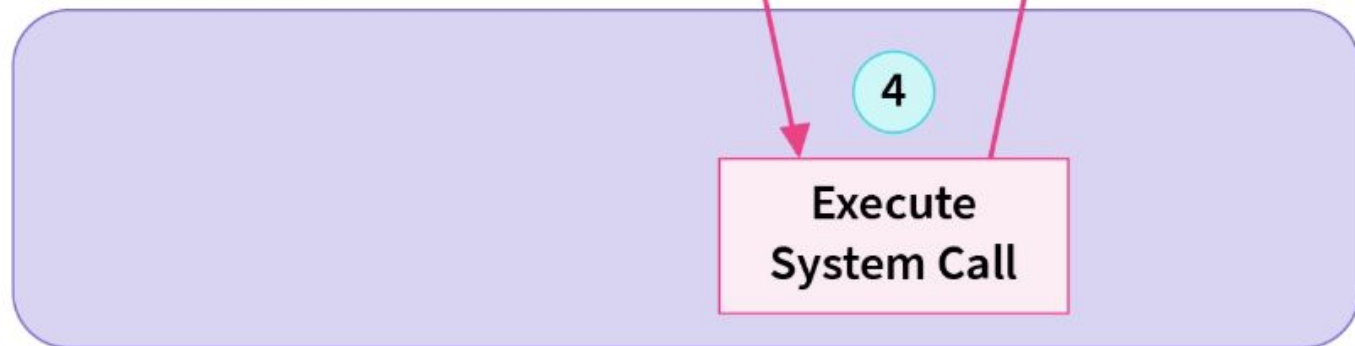


WORKING OF A SYSTEM CALL

USER MODE

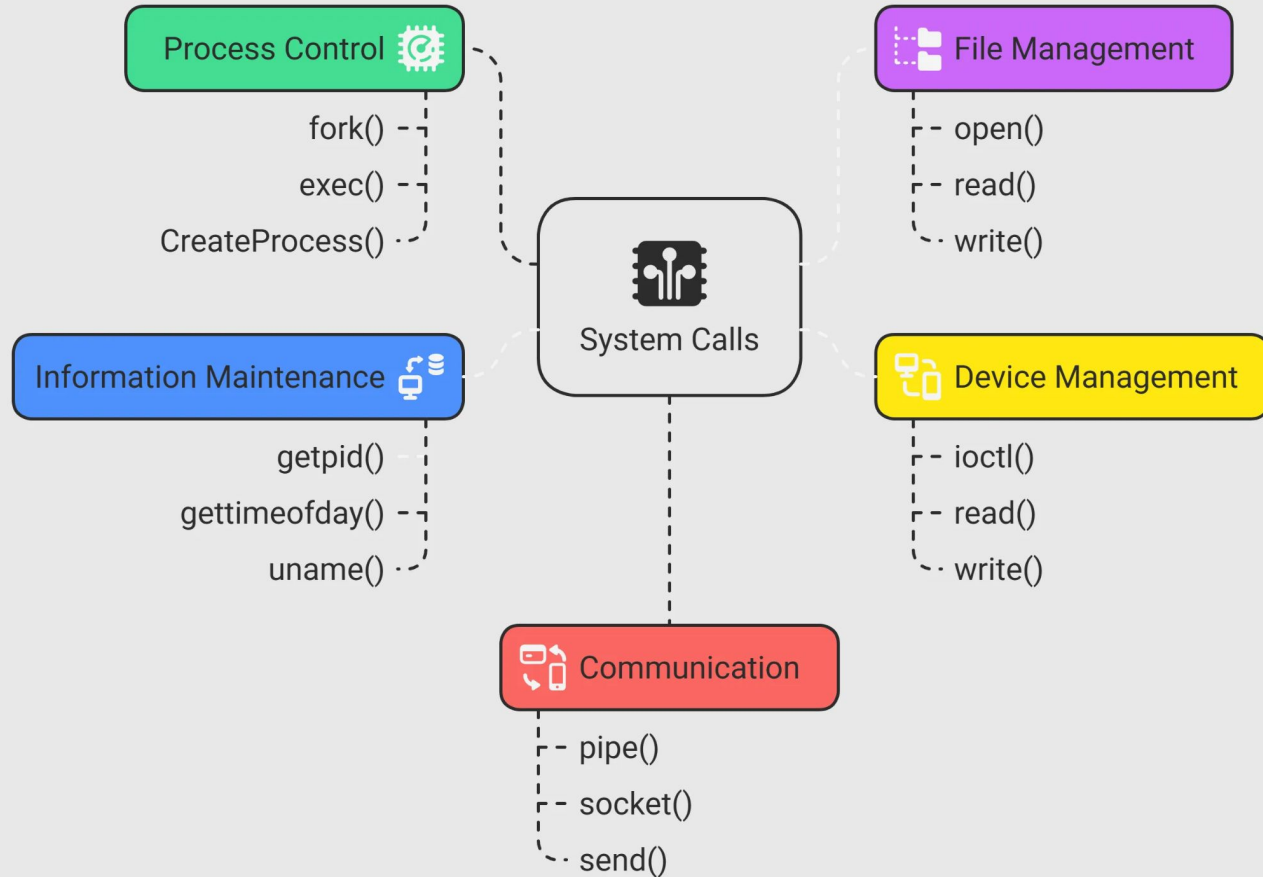


KERNEL MODE



System calls

System Call Types and Their Applications



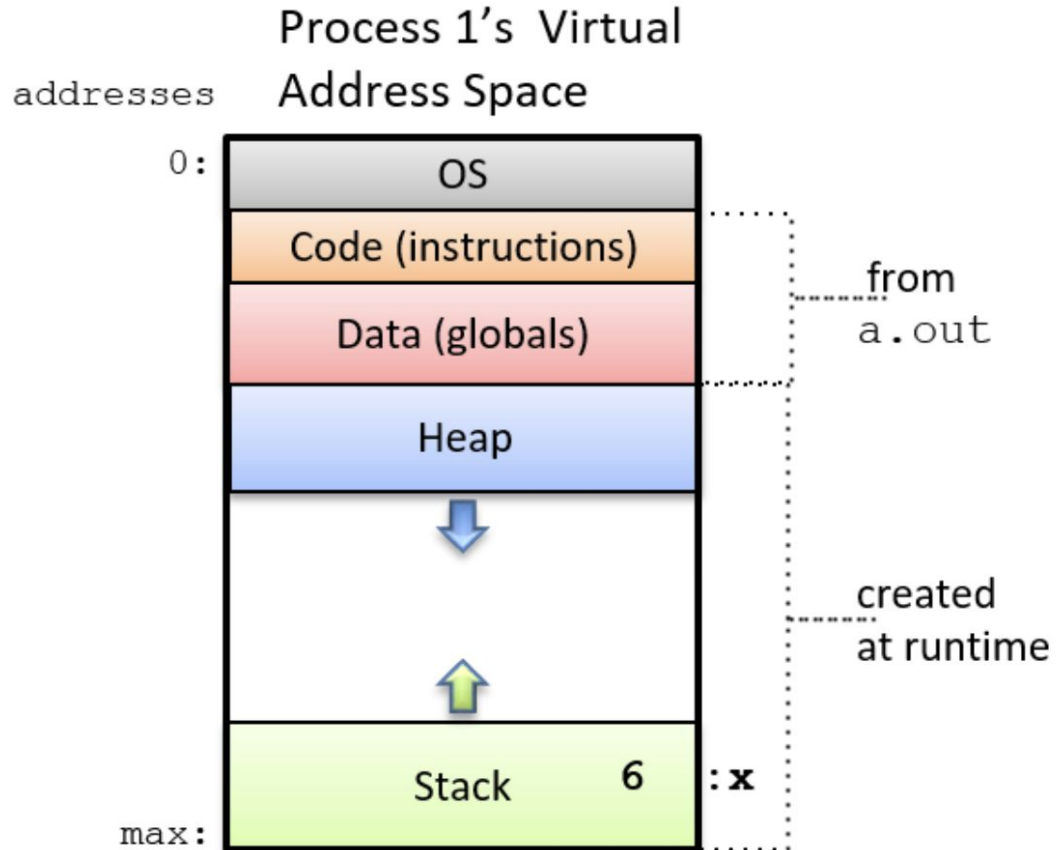
Process

- A process is a program that is currently being executed by the OS.
- When you run an exe file, the OS creates a process = a running program
- OS timeshares CPU across multiple processes: **virtualizes CPU**
- OS has a CPU scheduler that picks one of the many active processes to execute on a CPU

What Constitutes a Process?

- Every process has a unique process identifier (PID)
- Process occupies some memory in RAM (memory image)
 - *Code+data from executable*
 - *Stack+heap allocated for runtime use*
 - *Stack → function execution;*
 - *heap → dynamic allocation.*
- The execution context of the process (values of CPU registers)
 - *PC has address of instruction of process, some registers have process data*
 - *Process context is in CPU registers when process is running on CPU*
 - *Context saved in memory when process is paused, restored when run again*
- Ongoing communication with I/O devices
 - *Information is maintained about files that are open, ongoing network connections, other active connections to I/O devices*

Virtual address space



Execution context of the process (values of CPU registers)

1. **Program Counter (PC)**
 - a. Stores the address of the **next instruction** the process should execute. This tells the CPU where to continue.
2. **General-purpose registers**
 - a. Hold temporary process data, intermediate calculation results, function arguments, pointers, etc.
3. **Stack Pointer (SP)**
 - a. Points to the current position of the process's **stack**, which is needed to resume function calls correctly.
4. **CPU Flags / Status Register**
 - a. Stores CPU state such as zero, carry, sign, overflow, etc. Required to continue conditional instructions correctly.
5. **Process Control Block (PCB)**
 - a. OS data structure containing the process's saved CPU context along with other process information such as state, PID, scheduling information, etc.

How is a Program Loaded?

1. Program exists on disk
 - a. *The executable file contains machine code, data, metadata, and an entry point.*
2. User executes the program
3. OS creates a process
 - a. *The OS creates process-related information such as:*
 - i. PID
 - ii. **Process state**
 - iii. CPU registers
 - iv. File descriptors
 - v. Scheduling information
4. Virtual address space is created
5. Executable is loaded/mapped
 - a. *The OS loader maps the program's code and data into the process's virtual address space.
Required shared libraries are also mapped.*
6. Process becomes READY
7. CPU executes it

Summary & Important Questions

Summary

1. What is OS? Functions of OS?
2. Virtualization vs Abstraction
 - a. *VM vs Containerization*
3. System Call
 - a. *User mode vs Kernel mode*
4. What is Process? What Constitutes a Process?
5. Execution context of the process?
6. How is a Program Loaded?

Summary

1. What is OS? Functions of OS?

- a. *An Operating System (OS) is system software that manages hardware/resources and provides services and an interface for applications; its key functions include process, memory, file, I/O, storage, security, and resource management.*

2. Virtualization vs Abstraction

- a. *Abstraction hides complexity, while virtualization creates a virtual version of a physical resource so multiple isolated environments can share it.*
- b. *VM vs Containerization*
 - i. *A VM virtualizes the entire machine including the OS/kernel, whereas a container virtualizes the application environment while sharing the host OS kernel.*

3. System Call

- a. *A system call is the controlled interface through which a user-space program requests services from the OS kernel, such as `read()`, `write()`, or `fork()`.*
- b. *User mode vs Kernel mode*
 - i. *User mode restricts applications from directly accessing privileged resources, while kernel mode gives the OS full access to hardware and system resources.*

- What is Process? What Constitutes a Process?
 - *A process is a program in execution, consisting of its code, data, stack, heap, CPU execution context, and OS-managed metadata such as the PCB.*
- Execution context of the process?
 - *The execution context is the CPU state required to resume a process, mainly including the Program Counter, CPU registers, stack pointer, and processor status.*
- How is a Program Loaded?
 - *The OS creates a process, creates its virtual address space, maps/loads the program's code and data, initializes the stack and heap, sets the Program Counter to the entry point, and starts execution.*

States of Process

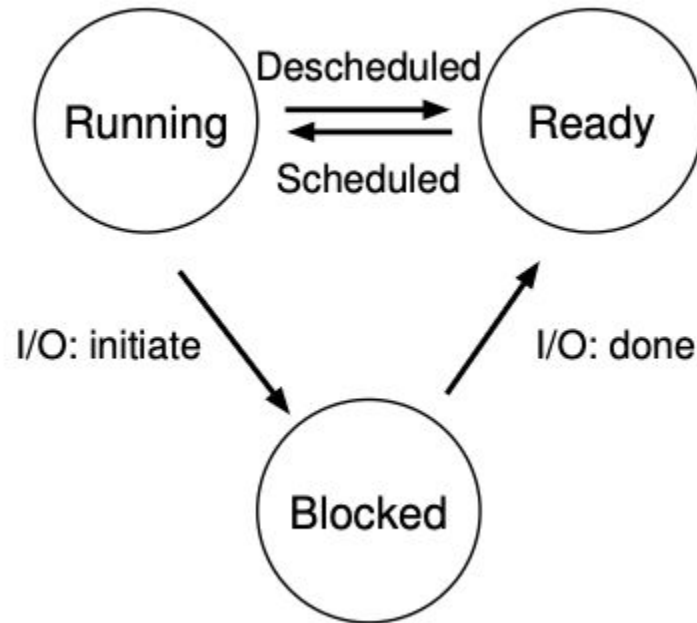
3 States of a process

OS manages multiple active processes at the same time. An active process can be in one of the following situations.

1. **Running:** currently executing on CPU
 - a. *CPU registers contain context of process*
2. **Blocked/suspended/sleeping:** process cannot run for some time
 - a. *Example: process has requested data from disk, command issued, but process cannot proceed until the data from disk is available*
3. **Ready/runnable:** ready to run but waiting for OS scheduler to switch the process in
 - a. *Many processes can be ready but scheduler can only run one on a CPU core*

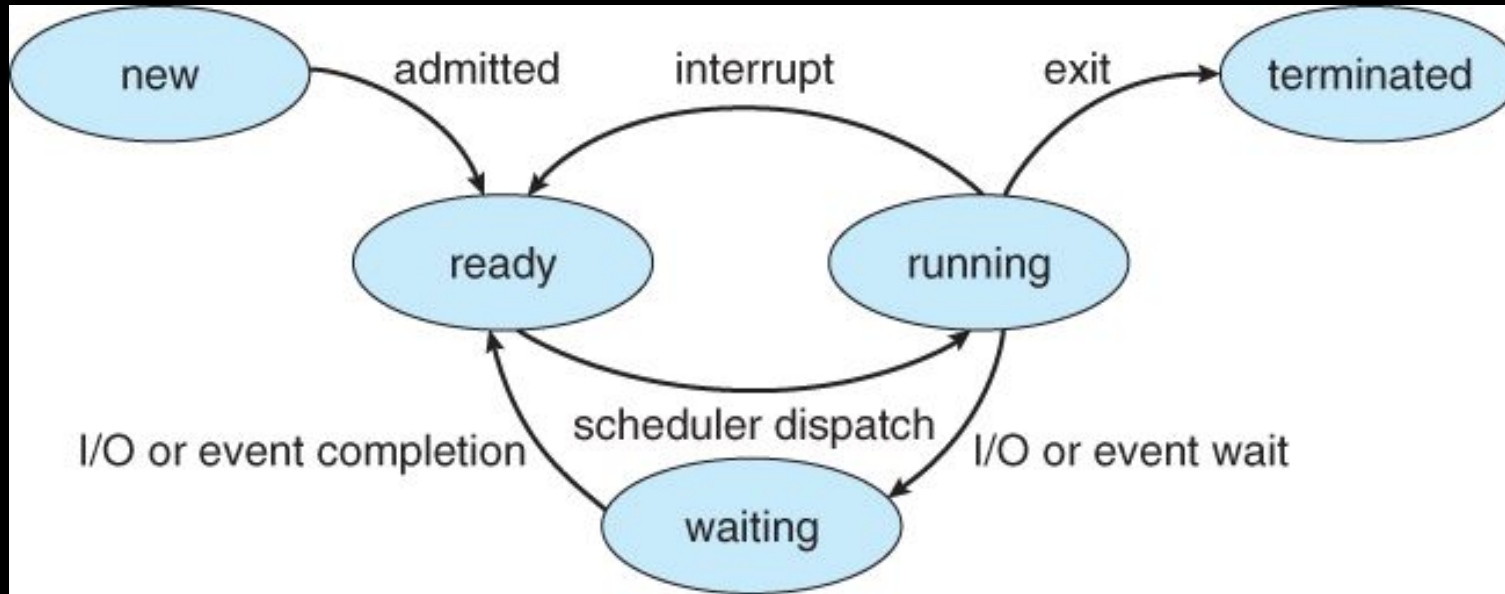
Context of blocked and ready processes is saved in OS memory, so that they can continue to run later on

3 state process model



5-state process model

- Important: Refer this model for exams, GATE Preparation



5-state process model

State	Meaning
New	Process is being created
Ready	Ready to execute, waiting for CPU
Running	Currently executing
Waiting/Blocked	Waiting for I/O/event
Terminated	Finished execution

GATE CSE 2024 — Process States

Question: Which of the following process state transitions is/are NOT possible?

- A. Running $\rightarrow$ Ready
- B. Waiting $\rightarrow$ Running
- C. Ready $\rightarrow$ Waiting
- D. Running $\rightarrow$ Terminated

GATE CSE 2024 — Process States

READY

"I am ready, but waiting for CPU."

RUNNING

"I currently have CPU."

WAITING

"I cannot proceed until some event/I/O happens."

TERMINATED

"I am finished."

Answer: B, C

GATE CSE 2020 — Process States

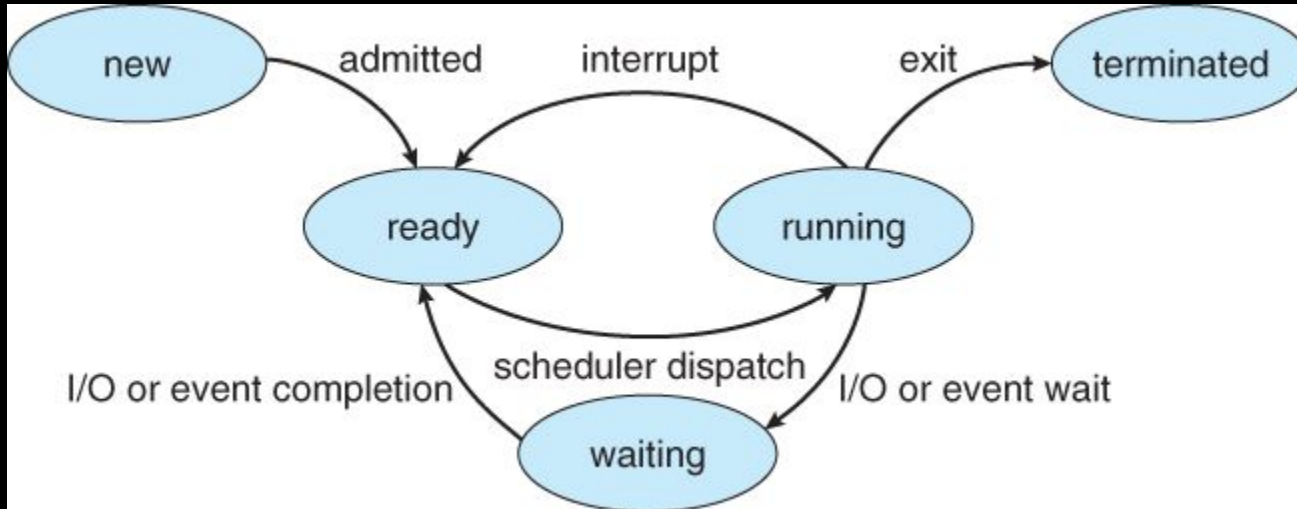
Consider the following statements about process state transitions for a system using preemptive scheduling.

- I. A running process can move to ready state.
- II. A ready process can move to running state.
- III. A blocked process can move to running state.
- IV. A blocked process can move to ready state.

GATE CSE 2020 — Process States

Answer: I, II and IV only

Always: Blocked (Waiting) → Ready → Running

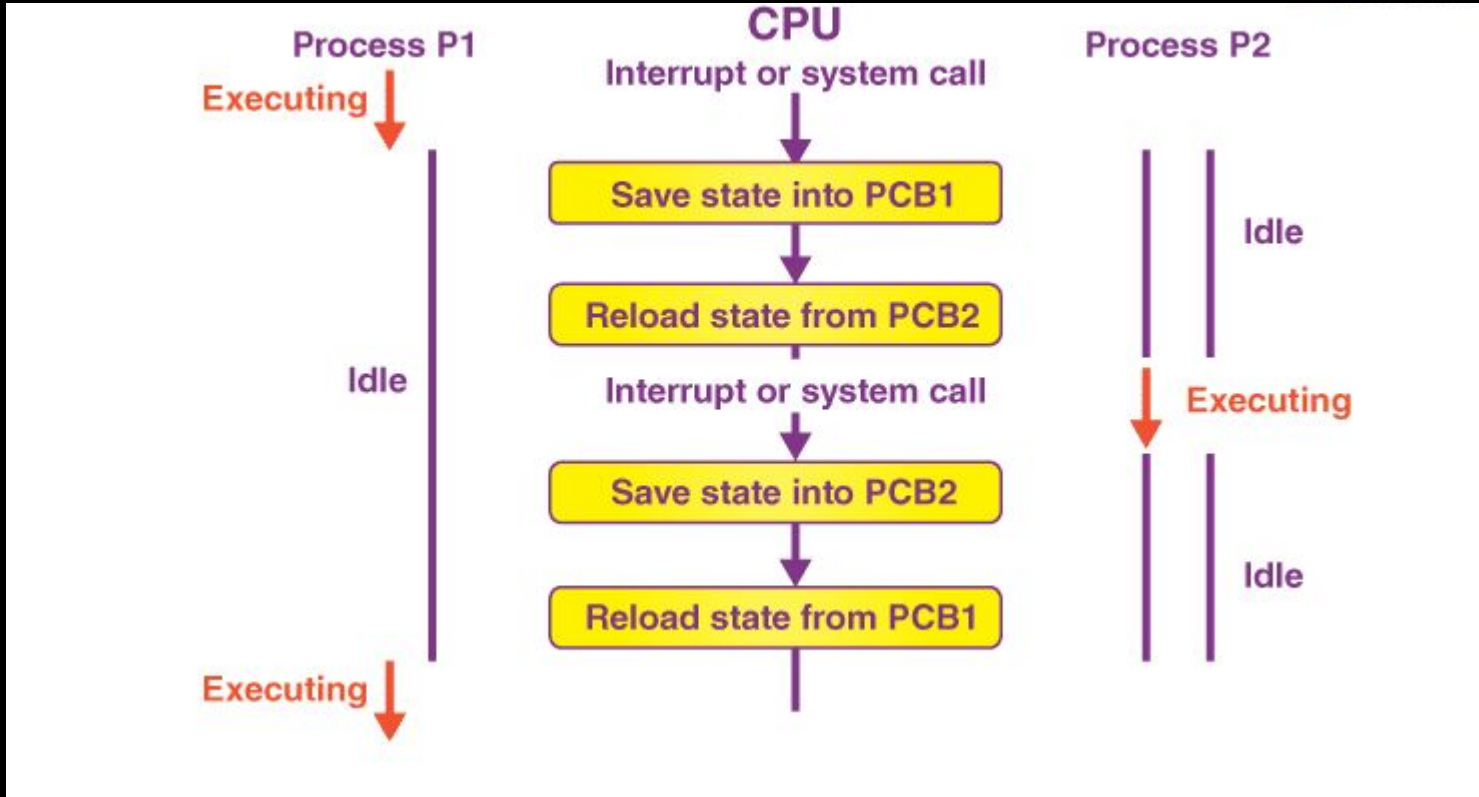


Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked,
5	Blocked	Running	so Process ₁ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Figure 4.4: Tracing Process State: CPU and I/O

Context Switch

Context switch = switching the CPU from one process/thread to another.



What happens during a context switch?

- The entire process is NOT copied during a context switch.
- Code, heap, stack, and data generally remain in memory.
- The OS mainly switches the CPU execution context.
- Switching between processes may also require changing the memory/address-space context.

Context Switch vs Mode Switch

Context Switch

Process/Thread A → B

Changes execution context

Performed by OS scheduler

Saves/restores CPU state

Has scheduling overhead

Mode Switch

User Mode → Kernel Mode

Changes CPU privilege level

Caused by system call, interrupt, exception

Same process can continue

Does not necessarily switch process

GATE 2011 — Mode Switch vs Context Switch

Let T_1 be the time taken to switch from user mode to kernel mode, and T_2 be the time taken to switch from one user process to another user process. What is the relationship?

A. $T_1 > T_2$

B. $T_1 = T_2$

C. $T_1 < T_2$

D. Cannot be determined

GATE 2011 — Mode Switch vs Context Switch

Answer: C. $T_1 < T_2$

Why is $T_1 < T_2$?

- Because a mode switch requires less state to change.
- For mode switch, the CPU changes privilege level and enters the kernel through the appropriate mechanism. The same process remains active.
- For context switch, the OS has to switch the execution context. This is more work

Process Control Block

All information about a process is stored in a kernel data structure called the process control block (PCB)

- • Process identifier (PID)
- • Process state (running, ready, blocked, terminated, ..)
- • Pointers to other related processes (parent, children)
- • Saved CPU context of process when it is not running
- • Information related to memory locations of a process
- • Information related to ongoing I/O communication

PCB is known by different names in different OS

- • struct proc in xv6
- • task_struct in Linux

CPU Scheduler

Every OS maintains list of PCBs in some data structure

- Array, linked list, heap – what is suitable when?
- Scheduler loops through list of PCBs and picks process to run, switches to process, switches to another process after some time, and this continues...
- Scheduler picks one process to run on every core, so number of running process is the number of parallel processors available

GATE CSE 2023

Question: When switching from thread T1 to thread T2 of the same process, which need to be saved?

1. Page table base register
2. Stack pointer
3. Program counter
4. General-purpose registers

GATE CSE 2023

Answer: 2, 3, 4

Process vs Thread

1. Processes are usually **independent**, while threads exist as **subsets of a process**.
2. Each process has its own memory space. Threads that belong to the same process **share the same memory**.
3. A process is a **heavyweight operation**. It takes more time to create and terminate.
4. Context switching is more expensive between processes.
5. Inter-thread communication is faster for threads.

Process

